Construindo uma aplicação com autenticação por sessão

Spring Boot • Spring Security • Thymeleaf • JPA • H2

Apostila prática em português | Edição de 20 de outubro de 2026

Do projeto Maven às telas personalizadas de login e logout, com gerenciamento de usuários e testes de integração.

Material elaborado a partir dos arquivos presentes no projeto session_based nesta data. O objetivo é reproduzir o estado atual da aplicação. As observações sobre limitações explicam o que ainda precisa evoluir; não representam funcionalidades já implementadas.

Sumário

1. O que você vai construir 3

2. Entendendo a sessão antes do código 3

3. Preparar o ambiente e criar o projeto 4

- 3.1 Dependências Maven . . . 4
- 3.2 Configuração do banco . . . 6

4. Modelar o usuário e criar o repositório 6

- 4.1 Entidade Usuario . . . 6
- 4.2 Repositório UsuarioRepository . . . 7

5. Criar o ponto de entrada e o usuário inicial 7

6. Configurar a autenticação e as regras de acesso 8

7. Implementar navegação e gerenciamento de usuários 10

8. Construir a página inicial 12

9. Construir o formulário e a lista de usuários 13

10. Personalizar o login e o logout 15

- 10.1 Estilos compartilhados . . . 15
- 10.2 Tela de login . . . 15
- 10.3 Confirmação de logout . . . 16

11. Executar e conferir no navegador 16

12. Adicionar e executar os testes de integração 17

- 12.1 Comandos de teste e empacotamento . . . 18

13. Diagnosticar problemas frequentes 18

14. Limites do ponto atual e próximas evoluções 19

15. Revisão guiada 20

- Referências oficiais . . . 20

1. O que você vai construir

Ao final, você terá uma aplicação web que exige autenticação, mostra o nome do usuário conectado, permite cadastrar, listar, editar e excluir usuários e oferece telas próprias de entrada e confirmação de saída. As senhas são armazenadas como hashes BCrypt e o login é mantido por sessão HTTP.

Pré-requisitos de conhecimento: fundamentos de Java, classes e interfaces, HTML e requisições HTTP. Não é necessário conhecer previamente Spring Security; seus componentes serão apresentados durante a construção.

A sequência didática desta apostila organiza a implementação por dependências. Ela não pretende reconstruir a ordem histórica dos commits. Cada listagem corresponde ao arquivo completo atual, salvo os exemplos de comandos e trechos explicitamente identificados como explicativos.

Roteiro: preparar o ambiente; criar o projeto; configurar o banco; modelar o usuário; criar o repositório; inicializar o administrador; configurar a segurança; implementar o controlador; construir as telas; executar; testar e revisar as limitações.

2. Entendendo a sessão antes do código

HTTP, isoladamente, não mantém a identidade de uma requisição para outra. No login, o Spring Security consulta o usuário, verifica a senha com o PasswordEncoder e estabelece o contexto de autenticação associado à sessão no servidor.

O navegador conserva o identificador da sessão em um cookie, normalmente JSESSIONID. Nas próximas requisições ele envia esse cookie, permitindo que o servidor recupere o contexto correspondente. A senha não precisa ser reenviada em cada navegação.

Fluxo de entrada: navegador acessa uma rota protegida → servidor redireciona para /login → formulário envia username, password e token CSRF → segurança valida as credenciais → navegador passa a acessar as rotas protegidas com a sessão autenticada.

Fluxo de saída: GET /logout apresenta a confirmação → POST /logout com CSRF encerra a sessão → servidor limpa a autenticação, solicita a remoção do cookie e redireciona para /login?logout.

BCrypt produz um hash de senha; ele não é uma criptografia reversível. Para verificar a senha, o encoder compara a tentativa com o hash armazenado. Não existe uma etapa de “descriptografar” a senha.

CSRF protege requisições que alteram estado contra o uso indevido de credenciais enviadas automaticamente pelo navegador. Os formulários POST com th:action recebem o campo oculto do token pela integração com Thymeleaf. Mantenha essa proteção habilitada. [1, 2]

Duas abas do mesmo navegador normalmente compartilham o cookie e a sessão. Sair em uma aba encerra o acesso autenticado nas próximas requisições da outra; uma página já exibida pode continuar visualmente aberta até ser atualizada.

3. Preparar o ambiente e criar o projeto

Use JDK 26: o pom.xml atual define java.version como 26. O projeto usa Spring Boot 4.1.0 e Maven Wrapper configurado para Maven 3.9.16. Essas são as versões do repositório, sem substituí-las por outras versões nesta apostila.

Confira o Java e o Maven Wrapper no terminal, dentro da pasta do projeto:

```
java -version
./mvnw -version
```

No Windows, utilize mvnw.cmd no lugar de ./mvnw. A primeira execução do wrapper pode exigir acesso à internet para baixar Maven e dependências. Se o script não tiver permissão de execução no Linux ou macOS, use chmod +x mvnw.

Para iniciar do zero, crie um projeto Java/Maven na IDE ou no Spring Initializr (https://start.spring.io), com Group com.example.sessionbased, Artifact session_based e pacote com.example.sessionbased.session_based. Escolha as dependências Spring Web, Thymeleaf, Spring Security, Spring Data JPA e H2. Confira o pom.xml abaixo e inclua também a dependência de testes de segurança.

Se a ferramenta gerar outro pacote ou outra versão, ajuste os arquivos para os valores desta apostila antes de copiar as classes. Preserve os arquivos mvnw, mvnw.cmd e .mvn/wrapper gerados com o projeto. Para reproduzir o wrapper atual, a propriedade distributionUrl aponta para Maven 3.9.16.

Todas as classes principais ficam no mesmo pacote. Isso permite que o component scan da classe principal encontre controladores, configuração, entidade e repositório.

```
session_based/
  pom.xml
  mvnw / mvnw.cmd
  .mvn/wrapper/maven-wrapper.properties
  src/main/java/com/example/sessionbased/session_based/
    DemoApplication.java
    Usuario.java
    UsuarioRepository.java
    SecurityConfig.java
    HomeController.java
  src/main/resources/
    application.properties
    static/logo.png
    static/logobranco.png
    static/css/auth.css
    templates/home.html
    templates/cadastro.html
    templates/login.html
    templates/logout.html
  src/test/java/com/example/sessionbased/demo/
    DemoApplicationTests.java
```

3.1 Dependências Maven

Substitua o pom.xml pelo conteúdo abaixo. O parent gerencia versões compatíveis das dependências; o plugin do Spring Boot usa DemoApplication como ponto de entrada.

Spring Web atende as requisições MVC; Thymeleaf renderiza HTML no servidor; Security aplica os filtros de autenticação; Data JPA oferece persistência; H2 é o banco embarcado. A dependência spring-boot-starter-security-test dá suporte aos testes usados neste projeto.

Arquivo: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example.sessionbased</groupId>
    <artifactId>session_based</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>session_based</name>
    <description>Demonstração do Session Based no Springboot</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>26</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <mainClass>com.example.sessionbased.session_based.DemoApplication</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

3.2 Configuração do banco

No estado atual, application.properties contém apenas comentários: não há URL de banco, usuário ou senha configurados ativamente. Com H2 disponível, o Spring Boot pode autoconfigurar um banco embarcado em memória. A inicialização automática do esquema depende dessa configuração e das ferramentas de migração presentes. [3]

Isso basta para a demonstração, mas os usuários cadastrados não devem ser considerados persistentes após reiniciar o processo. A aplicação não configura banco em arquivo nem servidor externo. Descomentar ddl-auto=update, isoladamente, não transforma um banco em memória em banco persistente.

As antigas linhas spring.security.user.name e spring.security.user.password estão comentadas. A autenticação efetiva virá do banco por meio do UserDetailsService, não dessas propriedades.

Crie o arquivo a seguir. Os comentários registram opções de geração de esquema, sem ativá-las.

Arquivo: src/main/resources/application.properties

```
#spring.application.name=demo
#spring.security.user.name=admin
#spring.security.user.password=123456
#spring.jpa.hibernate.ddl-auto=none


#update: Cria as tabelas caso elas não existam e atualiza a estrutura se você adicionar novas
    colunas.
    #(Não deleta colunas antigas).

#create: Sempre cria as tabelas ao iniciar, mas deleta as tabelas existentes primeiro (cuidado,
    você perde os dados).

#create-drop: Cria as tabelas e as deleta assim que a aplicação for encerrada.

#validate: Apenas valida se as tabelas já existem e se batem com a sua classe @Entity.
#Se não existirem, o sistema joga um erro e não inicia.

#none: Não faz nada. É o padrão para ambientes de produção.
```

4. Modelar o usuário e criar o repositório

4.1 Entidade Usuario

Crie Usuario.java no pacote principal. @Entity torna a classe uma entidade JPA; @Id identifica a chave primária; @GeneratedValue delega a geração do identificador ao banco. Os demais campos representam os dados do usuário.

O campo senha armazena o hash BCrypt. O nome do campo não implica armazenamento em texto puro: a transformação é feita antes de salvar. Os getters e setters permitem que JPA, controlador e formulário acessem os atributos.

Arquivo: src/main/java/com/example/sessionbased/session_based/Usuario.java

```
package com.example.sessionbased.session_based;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Usuario {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private String email;
    private String username;
    private String senha;

    // Getters e Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }
}
```

4.2 Repositório UsuarioRepository

Crie a interface a seguir. JpaRepository<Usuario, Long> informa o tipo de entidade e de seu identificador. O Spring Data cria a implementação durante a inicialização; você não precisa escrever SQL para save, findAll, findById ou deleteById.

Verificação: confira se a entidade e o repositório estão sob o pacote da aplicação principal. Caso contrário, a descoberta automática pode não encontrá-los.

Arquivo: src/main/java/com/example/sessionbased/session_based/UsuarioRepository.java

```
package com.example.sessionbased.session_based;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    // O Spring Data JPA já cria os métodos save(), findAll(), findById() automaticamente
}
```

5. Criar o ponto de entrada e o usuário inicial

Crie DemoApplication.java. @SpringBootApplication habilita a configuração e a descoberta dos componentes. O método main inicia a aplicação.

O bean CommandLineRunner roda após a inicialização. Quando o repositório inteiro está vazio, cria Administrador, com e-mail admin@gmail.com, login admin e senha de demonstração 123456. A senha é convertida para BCrypt antes de salvar.

A condição é repository.count() == 0: ela verifica se existem quaisquer usuários. Não procura especificamente por admin e não recria esse login se o banco tiver outros registros. Use as credenciais de demonstração apenas neste exercício local.

Arquivo: src/main/java/com/example/sessionbased/session_based/DemoApplication.java

```
// ... Ajuste sua classe DemoApplication para ficar assim:
package com.example.sessionbased.session_based;
```

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @Bean
    CommandLineRunner init(UsuarioRepository repository) {
        return args -> {
            // Cria um usuário mestre inicial salvo diretamente no banco com senha criptografada
            if(repository.count() == 0) {
                Usuario admin = new Usuario();
                admin.setNome("Administrador");
                admin.setEmail("admin@gmail.com");
                admin.setUsername("admin");
                admin.setSenha(new BCryptPasswordEncoder().encode("123456"));
                repository.save(admin);
            }
        };
    }
}
```

6. Configurar a autenticação e as regras de acesso

Crie SecurityConfig.java com três beans: SecurityFilterChain, PasswordEncoder e UserDetailsService. A configuração completa abaixo é o estado final, já com telas personalizadas.

SecurityFilterChain libera /login, /css/**, as duas imagens e /error. Qualquer outra requisição precisa estar autenticada. Não há restrição por papel administrativo: o requisito é estar conectado.

formLogin define a página GET /login e o processamento POST /login. O filtro do Spring Security recebe username e password; não crie um @PostMapping de login no controlador. Falhas redirecionam para /login?error.

A chamada defaultSuccessUrl("/") fornece o destino padrão. Se houver uma requisição protegida salva antes do login, ela pode ter prioridade sobre esse destino. O código não usa a opção de forçar sempre a página inicial.

logout configura POST /logout para invalidar a sessão, limpar a autenticação, remover JSESSIONID e redirecionar para /login?logout. GET /logout será apenas a página de confirmação criada no controlador. [2]

UserDetailsService busca o login na lista retornada por findAll e converte a entidade para o User do Spring Security. Todos recebem o papel USER. Embora funcione nesta demonstração, carregar todos os registros a cada login deve ser substituído por uma consulta específica em uma evolução do projeto.

PasswordEncoder fornece BCrypt para verificar hashes. O controlador e o inicializador também instanciam BCryptPasswordEncoder diretamente; isso reproduz o código atual, embora centralizar o uso no bean seja uma melhoria possível.

Arquivo: src/main/java/com/example/sessionbased/session_based/SecurityConfig.java

```
package com.example.sessionbased.session_based;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // 1. Configura as regras de acesso (Quem pode ver o quê)
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/login", "/css/**", "/logo.png", "/logobranco.png",
    "/error").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .loginProcessingUrl("/login")
                .defaultSuccessUrl("/")
                .failureUrl("/login?error")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout") // POST com CSRF; GET é nossa tela de confirmação
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .clearAuthentication(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
            );

        return http.build();
    }

    // 2. Define o decodificador de senhas (BCrypt)
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // 3. Conecta o Spring Security ao nosso banco de dados real
    @Bean
    public UserDetailsService userDetailsService(UsuarioRepository repository) {
        return username -> {
            // Busca o usuário no banco pelo username digitado na tela de login
            Usuario usuario = repository.findAll().stream()
                .filter(u -> u.getUsername().equals(username))
                .findFirst()
                .orElseThrow(() -> new UsernameNotFoundException("Usuário não encontrado: " +
    username));

            // Retorna um objeto de usuário que o Spring Security entende
            return User.builder()
                .username(usuario.getUsername())
                .password(usuario.getSenha()) // Aqui já vai estar criptografada com BCrypt
                .roles("USER")
                .build();
        };
    }
}
```

7. Implementar navegação e gerenciamento de usuários

Crie HomeController.java. @Controller indica que os retornos de texto são nomes de templates ou instruções de redirecionamento. Retornar "home" faz o Thymeleaf renderizar templates/home.html; retornar "redirect:/..." provoca uma nova navegação do navegador.

Mapa de rotas do controlador e dos filtros:

GET /login: formulário público de entrada. POST /login: autenticação processada pelo Spring Security.

GET /: página inicial protegida. O Principal identifica o usuário autenticado; seu nome é enviado ao Model como username.

GET /usuarios/novo: prepara usuarioForm vazio e listaUsuarios com todos os registros.

POST /usuarios/cadastrar: salva um novo usuário ou atualiza um existente conforme a presença de id. Criação aplica BCrypt; edição com senha vazia preserva o hash anterior, quando o registro é encontrado; edição com nova senha aplica outro hash.

GET /usuarios/editar/{id}: carrega o usuário no formulário e mantém a lista na mesma tela. Se o registro não existir, redireciona com ?erro.

GET /usuarios/excluir/{id}: exclui o registro, exceto quando o identificador é 1. Essa regra protege um identificador fixo, não identifica quem está autenticado.

GET /logout: confirmação de saída. POST /logout: encerramento efetivo pelo filtro de segurança, com CSRF.

Os parâmetros ?sucesso, ?erro e ?excluido são enviados pelos redirecionamentos do cadastro, mas o template atual não apresenta mensagens específicas para eles.

Digite o arquivo completo abaixo. O capítulo de limitações explica os cuidados necessários com edição e exclusão antes de reutilizar este fluxo em outro contexto.

Arquivo: src/main/java/com/example/sessionbased/session_based/HomeController.java

```
package com.example.sessionbased.session_based;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import java.util.Optional;

@Controller
public class HomeController {

    @Autowired
    private UsuarioRepository usuarioRepository;

    private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();

    @GetMapping("/login")
    public String login() {
        return "login";
    }
```

```
    @GetMapping("/logout")
    public String confirmarLogout() {
        return "logout";
    }

    @GetMapping("/")
    public String paginaInicial(java.security.Principal principal, Model model) {
        model.addAttribute("username", principal.getName());
        return "home";
    }

    // Alinhado para listar todos os usuários na mesma tela
    @GetMapping("/usuarios/novo")
    public String exibirFormularioCadastro(Model model) {
        model.addAttribute("usuarioForm", new Usuario()); // Objeto vazio para o formulário
        model.addAttribute("listaUsuarios", usuarioRepository.findAll()); // Lista preenchida do
    banco
        return "cadastro";
    }

    // Recebe os dados e decide se Cria um novo ou Edita um existente
    @PostMapping("/usuarios/cadastrar")
    public String cadastrarUsuario(Usuario usuario) {
        if (usuario.getId() != null) {
            // Cenário de EDICAO: Busca o registro atual do banco
            Optional<Usuario> usuarioAntigoOpt = usuarioRepository.findById(usuario.getId());
            if (usuarioAntigoOpt.isPresent()) {
                Usuario usuarioAntigo = usuarioAntigoOpt.get();
                // Se o adm não digitou uma nova senha, mantém a senha antiga do banco
                if (usuario.getSenha() == null || usuario.getSenha().trim().isEmpty()) {
                    usuario.setSenha(usuarioAntigo.getSenha());
                } else {
                    // Se digitou uma nova senha, criptografa a nova senha
                    usuario.setSenha(encoder.encode(usuario.getSenha()));
                }
            }
        } else {
            // Cenário de NOVO CADASTRO: Criptografia obrigatória
            usuario.setSenha(encoder.encode(usuario.getSenha()));
        }

        usuarioRepository.save(usuario);
        return "redirect:/usuarios/novo?sucesso";
    }

    // Rota acionada ao clicar em "Editar" na tabela
    @GetMapping("/usuarios/editar/{id}")
    public String carregarDadosParaEdicao(@PathVariable("id") Long id, Model model) {
        Optional<Usuario> usuario = usuarioRepository.findById(id);
        if (usuario.isPresent()) {
            model.addAttribute("usuarioForm", usuario.get()); // Preenche o formulário com os dados
     do usuário encontrado
            model.addAttribute("listaUsuarios", usuarioRepository.findAll()); // Mantém a lista
    atualizada abaixo
            return "cadastro";
        }
        return "redirect:/usuarios/novo?erro";
    }

    // Rota acionada ao clicar em "Excluir" na tabela
    @GetMapping("/usuarios/excluir/{id}")
    public String excluirUsuario(@PathVariable("id") Long id) {
        // Evita que o administrador delete a si próprio por acidente (opcional, baseado no id 1 do
     admin)
        if (id != 1) {
            usuarioRepository.deleteById(id);
        }
        return "redirect:/usuarios/novo?excluido";
    }
}
```

8. Construir a página inicial

Crie templates/home.html. ${username} é o atributo enviado pelo controlador. th:text escreve seu valor como texto escapado; th:href e th:src montam URLs relativas ao contexto da aplicação.

O botão de cadastro leva à tela de gerenciamento. O botão de saída abre a confirmação, sem encerrar a sessão imediatamente.

Copie logo.png e logobranco.png do projeto para src/main/resources/static. Para reconstruir com sua própria identidade visual, coloque imagens com esses mesmos nomes ou ajuste suas referências. Sem as imagens, o fluxo de autenticação continua disponível, mas os logotipos não aparecem.

Arquivo: src/main/resources/templates/home.html

```
<!DOCTYPE html>
<html xmlns:th="http://thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Página Inicial - Session Based</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 30px;
            background-color: #f9f9f9;
        }
        .container {
            max-width: 600px;
            margin: 0 auto;
            padding: 20px;
        }
        .logo-canal {
            max-width: 200px; /* Ajuste o tamanho máximo do seu logotipo aqui */
            height: auto;
            margin-bottom: 20px;
        }
        .btn {
            display: inline-block;
            padding: 10px 20px;
            background-color: #ff4d4d;
            color: white;
            text-decoration: none;
            border-radius: 5px;
            font-weight: bold;
            margin-top: 20px;
        }
        .btn:hover {
            background-color: #cc0000;
        }
    </style>
</head>
<body>

    <div class="container">
        <!-- O Spring mapeia a pasta 'static' diretamente na raiz /, por isso usamos apenas
    /logo.png -->
        <img th:src="@{/logo.png}" alt="Logotipo do Canal" class="logo-canal">

        <h1> Login Efetuado com Sucesso!</h1>
        <p>Usuário: <strong th:text="${username}">usuário</strong></p>
        <p>Você está logado no sistema utilizando Autenticação Baseada em Sessão.</p>

        <a href="/usuarios/novo" style="display: inline-block; padding: 10px 20px;
    background-color: #007bff; color: white; text-decoration: none; border-radius: 5px;
    font-weight: bold; margin-top: 20px; margin-right: 10px;">
            Ir para Cadastro de Usuários
        </a>

        <a th:href="@{/logout}" class="btn">Efetuar Logout (Sair)</a>
    </div>
```

```
</body>
</html>
```

9. Construir o formulário e a lista de usuários

Crie templates/cadastro.html. O mesmo template atende inclusão e edição. th:object seleciona usuarioForm e th:field vincula cada campo à propriedade correspondente. O id oculto diferencia o registro novo do existente.

O formulário POST envia os dados para /usuarios/cadastrar; th:action permite incluir automaticamente o token CSRF. A lista usa th:each para repetir uma linha por usuário e th:text para mostrar os valores.

Para editar, o link leva o id na URL e o controlador devolve o mesmo template com os dados carregados. Para manter a senha na edição, deixe o campo de senha vazio. O comportamento concreto do campo vinculado deve ser conferido no HTML renderizado; uma evolução recomendada é usar um formulário próprio, sem enviar o hash armazenado à interface.

A exclusão atual é um link GET com confirmação JavaScript. A confirmação é apenas uma interação visual e não substitui autorização nem CSRF. Reproduzimos esse comportamento do projeto, mas ele precisa mudar para POST ou DELETE protegido em uma etapa posterior.

Arquivo: src/main/resources/templates/cadastro.html

```
<!DOCTYPE html>
<html xmlns:th="http://thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Gerenciador de Usuários - Interno</title>
    <style>
        body { font-family: Arial, sans-serif; margin-top: 30px; background-color: #f9f9f9;
    display: flex; flex-direction: column; align-items: center; }
        .container { width: 90%; max-width: 800px; background: white; padding: 25px; border-radius:
     8px; box-shadow: 0px 0px 10px rgba(0,0,0,0.1); margin-bottom: 20px; }
        .logo-canal { max-width: 120px; height: auto; display: block; margin: 0 auto 15px auto; }
        h2, p { text-align: center; }
        .form-cadastro { max-width: 400px; margin: 0 auto 30px auto; }
        .form-group { margin-bottom: 12px; text-align: left; }
        label { display: block; margin-bottom: 4px; font-weight: bold; color: #333; }
        input { width: 100%; padding: 8px; box-sizing: border-box; border: 1px solid #ccc;
    border-radius: 4px; }
        .btn-salvar { width: 100%; padding: 10px; background-color: #4CAF50; color: white; border:
    none; border-radius: 4px; font-weight: bold; cursor: pointer; font-size: 16px; }
        .btn-salvar:hover { background-color: #45a049; }

        /* Estilos da Tabela */
        table { width: 100%; border-collapse: collapse; margin-top: 20px; }
        th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
        th { background-color: #f2f2f2; font-weight: bold; }
        tr:nth-child(even) { background-color: #f9f9f9; }

        /* Botões de Ação */
        .btn-edit { background-color: #ff9800; color: white; padding: 6px 12px; text-decoration:
    none; border-radius: 4px; font-size: 13px; font-weight: bold; border: none; cursor: pointer; }
        .btn-edit:hover { background-color: #e68a00; }
        .btn-del { background-color: #f44336; color: white; padding: 6px 12px; text-decoration:
    none; border-radius: 4px; font-size: 13px; font-weight: bold; margin-left: 5px; }
        .btn-del:hover { background-color: #da190b; }
        .btn-voltar { display: block; text-align: center; margin-top: 15px; color: #0066cc;
    text-decoration: none; font-size: 14px; }
    </style>
</head>
<body>

    <div class="container">
        <img th:src="@{/logobranco.png}" alt="Logotipo do Canal" class="logo-canal">
        <h2> Gerenciador de Usuários</h2>
        <p style="color: #666; font-size: 14px;">Área Restrita Administrativa</p>

        <!-- Formulário de Cadastro/Edição -->
```

```
        <form th:action="@{/usuarios/cadastrar}" method="POST" class="form-cadastro"
    th:object="${usuarioForm}">
            <!-- Campo oculto para armazenar o ID na edição -->
            <input type="hidden" th:field="*{id}" id="id">

            <div class="form-group">
                <label for="nome">Nome Completo:</label>
                <input type="text" th:field="*{nome}" id="nome" required placeholder="Ex: João
    Silva">
            </div>

            <div class="form-group">
                <label for="email">E-mail:</label>
                <input type="email" th:field="*{email}" id="email" required placeholder="Ex:
    joao@email.com">
            </div>

            <div class="form-group">
                <label for="username">Nome de Usuário (Login):</label>
                <input type="text" th:field="*{username}" id="username" required placeholder="Ex:
    joao.silva">
            </div>

            <div class="form-group">
                <label for="senha">Senha:</label>
                <!-- Removido o 'required' para permitir editar o usuário sem obrigar a trocar a
    senha sempre -->
                <input type="password" th:field="*{senha}" id="senha" placeholder="Digite uma nova
    senha ou deixe em branco">
                <small style="color:#777;" id="senhaHelp"></small>
            </div>

            <button type="submit" class="btn-salvar" id="btnAcao">Salvar Usuário</button>
            <a th:if="*{id != null}" href="/usuarios/novo" class="btn-voltar" style="color:
    #555;">Cancelar Edição</a>
        </form>

        <hr style="border: 0; border-top: 1px solid #eee; margin: 30px 0;">

        <!-- Tabela de Visualização -->
        <h3> Usuários Cadastrados no Sistema</h3>
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Nome</th>
                    <th>E-mail</th>
                    <th>Login</th>
                    <th>Ações</th>
                </tr>
            </thead>
            <tbody>
                <!-- Loop que lê a lista enviada pelo Spring Controller -->
                <tr th:each="u : ${listaUsuarios}">
                    <td th:text="${u.id}">1</td>
                    <td th:text="${u.nome}">Nome</td>
                    <td th:text="${u.email}">email@email.com</td>
                    <td th:text="${u.username}">username</td>
                    <td>
                        <!-- Botão que recarrega a página passando o ID do usuário para edição -->
                        <a th:href="@{/usuarios/editar/{id}(id=${u.id})}"
    class="btn-edit">Editar</a>
                        <!-- Botão de exclusão rápida -->
                        <a th:href="@{/usuarios/excluir/{id}(id=${u.id})}" class="btn-del"
    onclick="return confirm('Deseja realmente excluir este usuário?')">Excluir</a>
                    </td>
                </tr>
            </tbody>
        </table>

        <a href="/" class="btn-voltar"><- Voltar para a Home</a>
    </div>

</body>
</html>
```

10. Personalizar o login e o logout

10.1 Estilos compartilhados

Crie static/css/auth.css. Ele centraliza o cartão, formata campos e botões e distingue mensagens de sucesso e erro. O caminho público será /css/auth.css, sem o prefixo /static.

Arquivo: src/main/resources/static/css/auth.css

```
* { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; display: grid; place-items: center; padding: 24px; background:
     #f4f6fa; color: #263247; font-family: Arial, sans-serif; }
.card { width: 100%; max-width: 420px; padding: 32px; border-radius: 12px; background: white;
    box-shadow: 0 8px 30px #20305015; }
h1 { margin-top: 0; font-size: 26px; }
p { line-height: 1.5; }
label { display: block; margin: 18px 0 6px; font-weight: bold; }
input { width: 100%; padding: 12px; border: 1px solid #8994a5; border-radius: 6px; font: inherit; }
button { width: 100%; margin-top: 24px; padding: 12px; border: 0; border-radius: 6px; background:
    #245dcc; color: white; font: inherit; font-weight: bold; cursor: pointer; }
button:hover { background: #19469f; }
a { color: #245dcc; }
.secondary { display: block; margin-top: 20px; text-align: center; }
.message { padding: 12px; border-radius: 6px; }
.error { background: #ffe9e9; color: #8e2020; }
.success { background: #e5f5e9; color: #205e32; }
```

10.2 Tela de login

Crie templates/login.html. Os nomes dos campos precisam ser username e password, esperados pelo filtro de login. O atributo name é o vínculo com a autenticação; o id atende rótulos e identificação no HTML.

param.error exibe a falha de credenciais; param.logout exibe a confirmação de saída. Esses parâmetros apenas controlam a mensagem da página. A presença de ?logout na URL, sozinha, não encerra sessão alguma.

O formulário usa method="post" e th:action="@{/login}". A integração inclui o token CSRF no HTML renderizado. Abrir o arquivo diretamente no disco não executa Thymeleaf: acesse a aplicação pelo servidor. [1]

Arquivo: src/main/resources/templates/login.html

```
<!DOCTYPE html>
<html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Entrar - Session Based</title>
    <link rel="stylesheet" th:href="@{/css/auth.css}">
</head>
<body>
<main class="card">
    <h1>Entrar no sistema</h1>
    <p>Informe seu usuário e senha para continuar.</p>
    <p th:if="${param.error}" class="message error" role="alert">Usuário ou senha inválidos.</p>
    <p th:if="${param.logout}" class="message success" role="status">Você saiu do sistema com
    sucesso.</p>
    <!-- Thymeleaf inclui o campo CSRF neste formulário POST. -->
    <form th:action="@{/login}" method="post">
        <label for="username">Usuário</label>
        <input id="username" name="username" type="text" autocomplete="username" required
    autofocus>
        <label for="password">Senha</label>
        <input id="password" name="password" type="password" autocomplete="current-password"
    required>
        <button type="submit">Entrar</button>
    </form>
```

```
</main>
</body>
</html>
```

10.3 Confirmação de logout

Crie templates/logout.html. O botão confirma com POST /logout; o link de continuar retorna à home e mantém a sessão. O GET dessa tela não deve invalidar a sessão. Essa distinção também é verificada nos testes.

Arquivo: src/main/resources/templates/logout.html

```
<!DOCTYPE html>
<html lang="pt-BR" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Sair - Session Based</title>
    <link rel="stylesheet" th:href="@{/css/auth.css}">
</head>
<body>
<main class="card">
    <h1>Deseja sair?</h1>
    <p>Confirme para encerrar sua sessão neste sistema.</p>
    <form th:action="@{/logout}" method="post">
        <button type="submit">Confirmar saída</button>
    </form>
    <a class="secondary" th:href="@{/}">Continuar no sistema</a>
</main>
</body>
</html>
```

11. Executar e conferir no navegador

Com os arquivos criados, abra o terminal na pasta que contém pom.xml e execute:

```
./mvnw spring-boot:run
```

No Windows:

```
mvnw.cmd spring-boot:run
```

Aguarde a mensagem de aplicação iniciada e acesse http://localhost:8080. Sem uma sessão autenticada, a aplicação deve redirecionar para o login. Entre com admin e 123456 se o banco tiver sido inicializado vazio.

Roteiro de verificação manual:

1. Abra / em uma janela privativa. Confira o redirecionamento para /login e o carregamento dos estilos. 2. Informe uma senha incorreta. Confira /login?error e a mensagem de credenciais inválidas. 3. Entre com admin / 123456. Confira seu nome na home. Se tiver solicitado antes uma rota protegida, o destino após o login pode ser essa rota. 4. Abra o cadastro e crie um usuário de teste, por exemplo Maria, maria@example.com, login maria e uma senha preenchida. Confira a nova linha na lista. 5. Edite o nome desse usuário e deixe a senha vazia. Depois saia e confira que a senha original ainda funciona. Para trocar a senha, edite novamente e preencha outra. 6. Faça login com o usuário criado. Observe que ele também consegue abrir o gerenciamento: o código atual não exige papel ADMIN. 7. Exclua apenas o usuário de teste durante este exercício. A regra atual impede excluir id 1, independentemente de quem estiver conectado. 8. Abra /logout e escolha continuar. Confira que permanece autenticado. Abra novamente e confirme a saída; a URL final deve conter /login?logout. 9. Atualize uma rota protegida após sair. Ela deve voltar a exigir login. Repita em outra aba do mesmo navegador para observar a sessão compartilhada.

Para verificar o cookie, abra as ferramentas do navegador e consulte armazenamento/cookies para localhost. Observe JSESSIONID, sem publicar seu valor. Na aba de rede, confira o POST de login e o POST de logout; o formulário também contém o campo _csrf.

Interrompa a aplicação com Ctrl+C. No modo atual em memória, reiniciar pode descartar os usuários criados e inicializar novamente a conta de demonstração.

12. Adicionar e executar os testes de integração

Crie a classe de teste no caminho abaixo. Seu pacote é com.example.sessionbased.demo, diferente do pacote principal; por isso @SpringBootTest(classes = DemoApplication.class) aponta explicitamente a aplicação que deve carregar.

MockMvc executa as requisições no contexto Spring sem abrir um navegador ou depender de uma porta HTTP externa. O setup aplica springSecurity para incluir a cadeia de filtros. A aplicação usa H2 e seu inicializador também roda neste contexto de teste.

O teste publicLoginAndProtectedHome verifica acesso protegido, tela pública, CSS, campo CSRF e mensagens de login. O teste authenticationAndLogoutInvalidateSharedSession verifica senha incorreta, login válido, uso da mesma sessão, confirmação de saída, rejeição de POST sem CSRF e invalidação com token válido.

O nome do segundo teste menciona sessão compartilhada porque reutiliza a mesma MockHttpSession entre requisições. Ele não automatiza duas abas reais de navegador.

Arquivo: src/test/java/com/example/sessionbased/demo/DemoApplicationTests.java

```
package com.example.sessionbased.demo;

import com.example.sessionbased.session_based.DemoApplication;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.mock.web.MockHttpSession;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.MockMvcBuilders;
import org.springframework.web.context.WebApplicationContext;

import static org.junit.jupiter.api.Assertions.*;
import static org.hamcrest.Matchers.containsString;
import static
    org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.csrf
    ;
import static
    org.springframework.security.test.web.servlet.setup.SecurityMockMvcConfigurers.springSecurity;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest(classes = DemoApplication.class)
class DemoApplicationTests {
    @Autowired WebApplicationContext context;
    MockMvc mvc;

    @BeforeEach
    void setup() {
        mvc = MockMvcBuilders.webAppContextSetup(context).apply(springSecurity()).build();
    }

    @Test
    void publicLoginAndProtectedHome() throws Exception {
        mvc.perform(get("/")).andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/login"));
        mvc.perform(get("/login")).andExpect(status().isOk())
```

```
            .andExpect(content().string(containsString("name=\"_csrf\"")))
            .andExpect(content().string(containsString("Entrar no sistema")));
        mvc.perform(get("/css/auth.css")).andExpect(status().isOk());
        mvc.perform(get("/login?error")).andExpect(content().string(containsString("Usuário ou
    senha inválidos")));
        mvc.perform(get("/login?logout")).andExpect(content().string(containsString("Você saiu do
    sistema com sucesso")));
    }

    @Test
    void authenticationAndLogoutInvalidateSharedSession() throws Exception {
        mvc.perform(post("/login").with(csrf()).param("username", "admin").param("password",
    "errada"))
            .andExpect(redirectedUrl("/login?error"));
        var result = mvc.perform(post("/login").with(csrf())
                .param("username", "admin").param("password", "123456"))
            .andExpect(redirectedUrl("/")).andReturn();
        var session = (MockHttpSession) result.getRequest().getSession(false);
        assertNotNull(session);
        mvc.perform(get("/").session(session)).andExpect(status().isOk())
            .andExpect(content().string(containsString("admin")));
        mvc.perform(get("/logout").session(session)).andExpect(status().isOk())
            .andExpect(content().string(containsString("Confirmar saída")))
            .andExpect(content().string(containsString("name=\"_csrf\"")));
        assertFalse(session.isInvalid());
        mvc.perform(post("/logout").session(session)).andExpect(status().isForbidden());
        assertFalse(session.isInvalid());
        mvc.perform(post("/logout").session(session).with(csrf()))
            .andExpect(redirectedUrl("/login?logout"))
            .andExpect(cookie().maxAge("JSESSIONID", 0));
        assertTrue(session.isInvalid());
        mvc.perform(get("/")).andExpect(status().is3xxRedirection());
    }
}
```

12.1 Comandos de teste e empacotamento

```
./mvnw test
./mvnw clean package
java -jar target/session_based-0.0.1-SNAPSHOT.jar
```

Execute os comandos em sequência, encerrando a instância anterior antes de iniciar o JAR na mesma porta. clean package recompila, executa os testes e gera o artefato. O modo -o do Maven só funciona se as dependências já estiverem disponíveis localmente.

O resultado esperado dos testes atuais é: 2 testes executados, nenhuma falha e nenhum erro. Os relatórios ficam em target/surefire-reports. Essa cobertura se concentra na autenticação; não comprova todos os casos do CRUD.

13. Diagnosticar problemas frequentes

Versão de Java incompatível: confirme java -version e ./mvnw -version. A IDE e o Maven precisam usar JDK 26, conforme o pom.xml. Uma versão diferente pode impedir a compilação.

Dependência não encontrada: confira acesso ao repositório Maven, proxy e disponibilidade das versões exatas do projeto. Não execute com -o antes de baixar as dependências.

Porta 8080 ocupada: encerre outra instância ou use o comando abaixo para executar temporariamente na porta 8081; ajuste a URL do navegador.

```
./mvnw spring-boot:run -Dspring-boot.run.arguments=--server.port=8081
```

Template não encontrado: confira nome e localização em src/main/resources/templates. O retorno do controlador não inclui a extensão .html.

CSS ou imagem com erro 404: confira os arquivos em static e as URLs /css/auth.css, /logo.png e /logobranco.png. A URL não deve conter /static.

POST com erro 403: confirme que o formulário foi renderizado pelo servidor e que o token CSRF e a sessão foram enviados. Nos testes, use .with(csrf()) quando o cenário exigir uma requisição válida.

Login recusado: confirme username, senha e criação do usuário inicial. Valores antigos em application.properties não substituem o UserDetailsService. Hashes BCrypt não devem ser digitados no campo de senha.

Dados sumiram ao reiniciar: o projeto não configura armazenamento persistente. O banco em memória atende esta demonstração; para preservar dados, será necessário configurar outra forma de armazenamento.

Testes não localizam a aplicação: preserve @SpringBootTest(classes = DemoApplication.class), já que a classe de teste está em outro pacote.

14. Limites do ponto atual e próximas evoluções

Esta seção descreve aspectos observados no código. Ela não altera a implementação reproduzida nas listagens.

Autorização: todos os usuários autenticados recebem USER e podem gerenciar usuários. O texto “Área Restrita Administrativa” é visual; não há verificação de ADMIN. Uma evolução deve definir papéis e aplicar regras de acesso no servidor.

Exclusão: a rota GET altera estado e não recebe a proteção CSRF aplicada aos POST. Troque esse fluxo por uma operação protegida em uma etapa futura. A comparação com id 1 também não garante impedir autoexclusão de outros usuários.

Validação: não há Bean Validation, unicidade de username ou e-mail nem tratamento completo de erros. O required do HTML pode ser contornado; o campo de senha não é obrigatório no cadastro. Uma evolução deve validar os dados no servidor e tratar duplicidades.

Edição: se o id informado não existir, a lógica não trata explicitamente esse caso antes de salvar. O objeto da entidade também é vinculado diretamente ao formulário. Use um modelo próprio de entrada e valide a existência do registro antes de atualizar.

Senhas: a edição coloca a entidade no Model, incluindo o hash armazenado. Evite expor esse atributo ao formulário e mantenha o campo de nova senha sempre vazio. Centralize o PasswordEncoder em vez de criar instâncias em vários pontos.

Persistência e consulta: H2 em memória perde dados ao encerrar; findAll para autenticar carrega a tabela inteira. Considere banco persistente, migrações e método de consulta por username, com índice e unicidade.

Testes: amplie a cobertura para criação, edição, exclusão, validação e autorização quando esses comportamentos evoluírem. Os testes atuais não garantem essas propriedades.

15. Revisão guiada

Antes de encerrar, explique com suas palavras: qual é a diferença entre senha, hash e cookie de sessão? Por que POST /login não precisa de um método no controlador? Por que GET /logout apenas mostra uma página? O que acontece com outra aba depois que a sessão é invalidada?

Exercício de leitura: localize no código quem salva o usuário inicial, quem transforma Usuario em UserDetails, quem adiciona username ao Model e quem gera as linhas da tabela. Depois acompanhe o caminho completo de um cadastro, desde th:field até usuarioRepository.save.

Checklist de conclusão: o projeto compila; /login abre sem autenticação; a home exige login; credenciais incorretas geram mensagem; um usuário pode ser criado e editado; a confirmação não encerra a sessão; o POST de logout exige CSRF e encerra a sessão; os dois testes passam.

Referências oficiais

As listagens foram extraídas dos arquivos locais do projeto em 20/10/2026. As referências abaixo apoiam os conceitos de autenticação, logout e banco embarcado. A documentação online pode evoluir; as versões reproduzidas no material são as definidas no pom.xml.

[1] Spring Security — Form Login.
https://docs.spring.io/spring-security/reference/7.0/servlet/authentication/passwords/form.html

[2] Spring Security — Handling Logouts.
https://docs.spring.io/spring-security/reference/servlet/authentication/logout.html

[3] Spring Boot — SQL Databases. https://docs.spring.io/spring-boot/reference/data/sql.html